

Gaussian Component-wise Boosting

Luca Messerschmidt Bjarne Herbst Benedict B. Schnitzler

2026-05-21

Overview

TODO: add citation

This vignette introduces the basic workflow for fitting Gaussian component-wise boosting models using **asp26boost**.

Boosting is a stage-wise optimization procedure that combines many simple model updates into a flexible prediction model. From a statistical perspective, boosting can be viewed as an iterative empirical risk minimization algorithm. Starting from an initial estimate, the model is repeatedly improved by fitting simple base learners to the current residual information and updating only the best-performing model component at each iteration. Early stopping acts as a form of regularization and often leads to sparse models by selecting only a subset of available predictors.

For Gaussian response models, component-wise boosting minimizes the squared-error loss. Beginning with an intercept-only model, each predictor is fitted separately to the current residuals. The predictor yielding the largest reduction in residual sum of squares is selected and updated by a shrunken step determined by the learning rate. Repeating this procedure yields coefficient estimates, variable selection, and regularization simultaneously.

The implementation provided by **asp26boost** follows this component-wise boosting framework and offers methods for model fitting, prediction, summarization, and visualization.

Installation

TBD TODO, for now only to be functional

Simulated Example Data

To illustrate the basic functionality, we generate a synthetic regression dataset with a sparse underlying signal.

```
set.seed(27092001)

n <- 1000 # number of observations
p <- 10   # number of covariates

X <- matrix(
  rnorm(n * p),
  nrow = n,
  ncol = p
)
```

```
colnames(X) <- paste0("x", seq_len(p))

beta <- c(2, -1.5, 1, rep(0, p - 3))

y <- X %*% beta + rnorm(n)

sample_data <- data.frame(
  y = as.numeric(y),
  X
)

head(sample_data)
```

	y	x1	x2	x3	x4	x5
1	1.9923565	-0.4261948	-1.1171738	0.48469532	1.2131425	-0.336170941
2	-0.7356261	-0.7131972	0.4245545	0.44002122	-0.8269670	-0.942629273
3	-5.4848620	-1.8385995	0.3599087	-0.22333695	-0.4738789	-0.996149133
4	-1.9246527	0.8625711	1.5398105	-0.07525713	1.4213003	-0.003867292
5	-3.2200384	-1.0810953	1.3918728	0.35499283	-0.5583812	0.978657121
6	-4.1853657	-1.7623401	-0.8189664	-1.73188716	-1.5627475	-0.983039525

	x6	x7	x8	x9	x10
1	-1.2355394	0.70395175	2.1222730	-1.7657446	0.4977308
2	0.8983855	-1.19335497	-0.9879921	0.2926119	0.8580337
3	-2.0669550	-0.65491769	0.5644572	0.5134706	0.3168015
4	0.4912211	0.22194407	0.7515952	-1.4363766	0.8185884
5	-0.7280939	0.04176014	0.3786812	-0.3892265	-1.1157866
6	-0.8300621	-2.22611065	0.9980038	0.3107517	-0.5029159

The response depends only on the first three predictors. The remaining covariates contain noise and are included to illustrate the variable-selection behavior of component-wise boosting.

Model Fitting

Gaussian component-wise boosting models are fitted using `boost_gaussian()`.

```
fit <- boost_gaussian(
  x = sample_data[, -1],
  y = sample_data$y,
  mstop = 100,
  nu = 0.1
)
```

The most important tuning parameters are:

- `mstop`: number of boosting iterations.
- `nu`: learning rate controlling the size of each boosting update.

Internally, predictors are standardized before fitting. The model is initialized with the mean response and updated iteratively by selecting the predictor that provides the largest reduction in squared-error loss.

Printing the fitted object provides a compact overview of the model.

```
fit

Gaussian Gradient Boosting
-----
Call:
```

```

boost_gaussian(x = sample_data[, -1], y = sample_data$y, mstop = 100,      nu = 0.1)

Tuning parameters:
  mstop : 100
  nu    : 0.1

Predictors:
  Total                : 10
  Selected (>= 1 time) : 5
  Never selected (coef=0): 5

Training fit:
  RSS      : 983.5307
  MSE      : 0.9835
  R-squared : 0.8829

Selected variables and selection counts:

x1  x2  x3 x10  x8
34  33  29   2   2

```

Typical output includes:

- the original model call
- the tuning parameters specified during the model call
- The number of predictors, including which were (not) selected
- basic training-fit statistics
- the names of the selected variables and selection frequencies

Model Summary

A more detailed summary can be obtained using `summary()`.

```

summary(fit)

Gaussian Gradient Boosting - Model Summary
-----
Call:
  boost_gaussian(x = sample_data[, -1], y = sample_data$y, mstop = 100,      nu = 0.1)

Dimensions : n = 1000  p = 10 ( 5 selected, 5 zero )

Iterations : mstop = 100  nu = 0.1

Residuals:
      Min       1Q   Median  Mean       3Q      Max
-3.6859 -0.6344 -0.0135   0  0.6459  3.0565

  RSS : 983.5307   MSE : 0.9835   R-squared : 0.8829

Coefficients on original predictor scale:
  Variable                Coefficient
-----
(Intercept)              0.0188

```

```

x1          1.9318
x2         -1.4602
x3          0.9355
x10         0.0106
x8          0.0097

```

Variables set to **zero** (never selected):
x4, x5, x6, x7, x9

The summary object contains information on:

- the original model call
- model dimensions
- tuning parameters
- summary of residuals
- basic training-fit statistics
- coefficient estimates on the original predictor scale for selected predictors
- list of non-selected predictors (`coefficient = 0`)

Prediction

Predictions can be computed using `predict()`.

```

predict(fit)
[1]  1.30604651 -1.56777746 -4.25862736 -0.61776896 -3.77814595 -3.80561131
[7]  1.64744990  2.03242024  0.77142643 -3.08876899 -6.36081742  1.79981255
[13]  4.14141877 -2.48655224 -4.22324504 -0.59695245 -3.68326464  0.56889764
[19]  0.99995549  3.54291188 -3.87322427  0.52620477 -0.85596499 -1.92450566
[25]  1.70972185  1.02626975 -0.91036853  2.84746278 -3.84824625 -4.40311894
[31]  0.30678587 -2.19165987  0.81071719  1.73067075  1.03821830 -1.62649813
[37]  2.72145893 -3.37251663  7.22225055  1.01325219  3.37937417  1.49841023
[43] -0.36465619  0.03268667 -0.10219133 -0.68981900 -0.83590132 -1.11769419
[49]  2.14689566 -0.77849245 -4.28913599  1.18776222  4.75963885  3.15747903
[55]  5.02804342 -1.62633365 -0.93244526  1.41486769 -0.58485649  2.91512846
[61] -7.33347507  3.08837216 -0.56321916 -3.44870930 -2.90818512 -3.86035051
[67]  3.48459072 -2.22084206 -0.67268887 -0.90422164 -2.42439797  1.06623327
[73]  0.08828299 -3.70819944 -0.48851881  2.57759658 -0.50459528  1.75227705
[79] -3.70706336 -6.72039149  2.89426114  0.07334461  1.35927821 -4.26421540
[85]  0.86180223  1.69716567  2.99721372  6.68831575  0.96146259  0.38937778
[91] -1.98649887 -3.37591893 -2.73596164  0.62743508 -2.67423862 -2.05801642
[97]  0.53965275 -3.57682882  0.12021454 -2.23855562
[ reached 'max' / getOption("max.print") -- omitted 900 entries ]

```

When no additional data are supplied, fitted values for the training sample are returned.

Predictions for new observations can be generated via the `newdata` argument.

```

new_data <- sample_data[1:5, -1]

predict(
  fit,
  newdata = new_data
)
[1]  1.306047 -1.567777 -4.258627 -0.617769 -3.778146

```

The prediction method also supports evaluating models at earlier boosting iterations.

```
predict(
  fit,
  newdata = new_data,
  mstop = 25
)
[1] 0.6521571 -1.2122798 -3.0129431 -0.2532270 -2.6702284
```

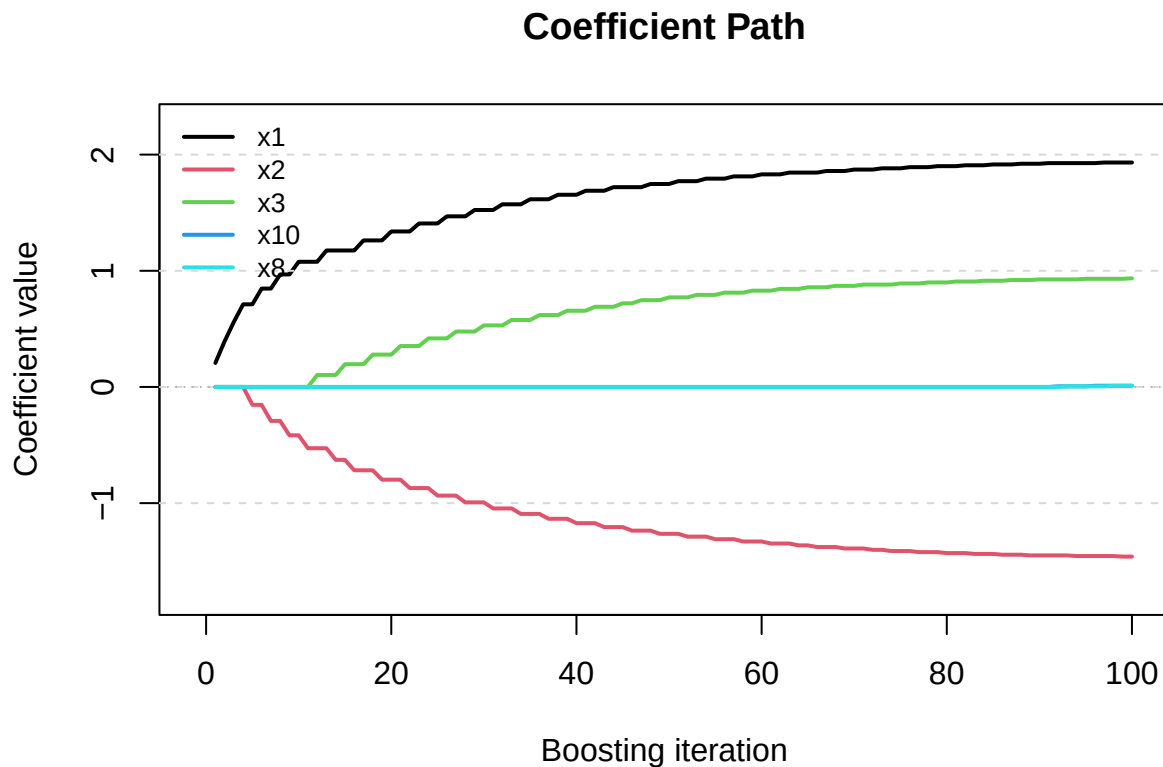
Note For in-sample early-stopped predictions, the model must have been fitted with `keep_path = TRUE`, which is the default when `keep_path` is unspecified.

Model Diagnostics and Visualization

The package provides graphical diagnostics through the `plot()` method.

Coefficient trajectories across boosting iterations can be visualized using the path plot.

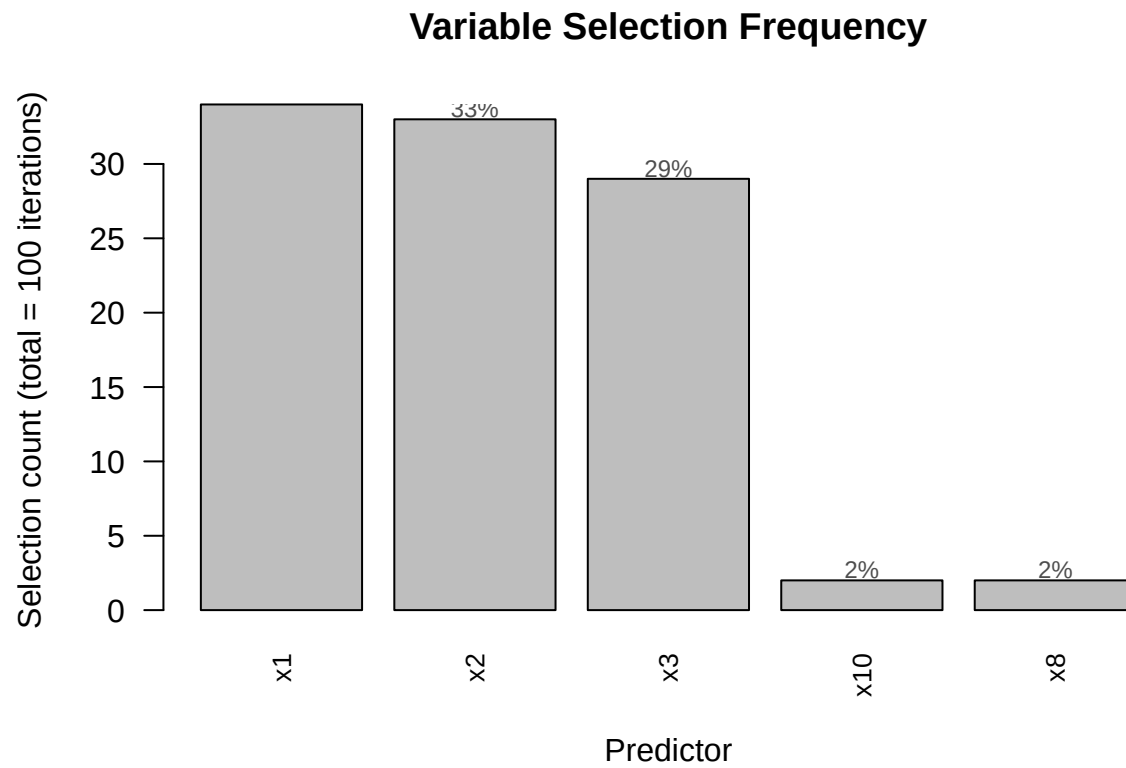
```
plot(
  fit,
  type = "path"
)
```



By default, the most frequently selected variables are displayed. Coefficients may be shown either on the original predictor scale or on the standardized training scale.

Variable selection frequencies can be inspected using

```
plot(  
  fit,  
  type = "frequency"  
)
```



This visualization summarizes how often each predictor was selected during the boosting process and provides a simple measure of variable importance.

Note To use the `plot()` diagnostics the model must be fitted with `keep_path = TRUE`, which is the default.